

ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

Filière Informatique – EMSI Casablanca

Rapport Scientifique Académique

Projet de Fin de Module – Deep Learning

Auteur :
Benebbou Ahmed

Encadrante :
Mme. Zineb Hidila

Table des matières

1	Introduction & Objectifs Généraux	2
2	Partie I : Perceptron Multicouche (MLP) & Ingénierie PyTorch	2
2.1	Concepts fondamentaux de PyTorch	2
2.2	Préparation des données & Architecture	2
2.3	Inspection des Paramètres	3
2.4	Impact des stratégies d'initialisation	3
2.5	Métriques d'évaluation sur le Test	4
2.6	Question de synthèse - Partie I	4
3	Partie II : Réseaux de Neurones Convolutionnels (CNN) & Vision	4
3.1	Limitations du MLP pour l'imagerie	4
3.2	Calcul dimensionnel et manuel	5
3.3	Implémentation Custom NumPy	5
3.4	Expérimentations d'Architectures sur Fashion-MNIST	5
3.5	Visualisation des cartes de caractéristiques	6
3.6	Comparaison Quantitative : MLP vs CNN	6
3.7	Question de synthèse - Partie II	6
4	Partie III : Modèles Séquentiels (RNN) & Seq2Seq	7
4.1	Modélisation probabiliste & Perplexité	7
4.2	BPTT & Rôle du Gradient Clipping	7
4.3	Modèle Seq2Seq pour la Traduction	7
4.4	Question de synthèse - Partie III	8
5	Question Transversale Finale	8
6	Conclusion	9

1 Introduction & Objectifs Généraux

Le deep learning (apprentissage profond) représente l'état de l'art dans la modélisation de relations non linéaires complexes sur des données massives. L'hypothèse fondamentale du deep learning est sa capacité à construire des représentations hiérarchiques de caractéristiques à partir de données brutes.

L'objectif de ce projet est de concevoir, implémenter et analyser des réseaux de neurones sous la bibliothèque PyTorch, adaptés aux trois formes géométriques de données majeures rencontrées dans l'industrie :

- **Données Tabulaires** : Classification de données biophysiques via un Perceptron Multicouche (MLP).
- **Données d'Images** : Classification de vêtements à l'aide de Réseaux de Neurones Convolutionnels (CNN).
- **Données Séquentielles** : Traduction automatique de séquences de texte par un modèle Encodeur-Décodeur récurrent (Seq2Seq).
-

2 Partie I : Perceptron Multicouche (MLP) & Ingénierie PyTorch

2.1 Concepts fondamentaux de PyTorch

Pour implémenter des architectures de deep learning, PyTorch fournit des briques fondamentales :

- **nn.Module** : Classe parente de tout modèle. Elle gère le graphe de calcul, l'enregistrement automatique des couches et permet de définir la propagation avant dans `forward()`.
- **Parameter** : Tenseurs spécifiques représentant les poids et les biais du réseau. Ils possèdent par défaut la propriété `requires_grad=True`.
- **Gradient** : Représente la matrice des dérivées partielles de la fonction de perte calculée par différentiation automatique (**Autograd**) lors de la rétropropagation.
- **state_dict** : Dictionnaire associant à chaque paramètre son tenseur de poids ou de biais. Il est utilisé pour sauvegarder l'état complet du modèle.
- **Device** : Spécification matérielle d'exécution (`cpu` ou `cuda` pour GPU).
- **Forward & Backward** : Le passage avant (*forward*) calcule la prédiction du modèle. La rétropropagation (*backward*) calcule les gradients à l'aide de la règle de dérivation des chaînes de fonctions.

2.2 Préparation des données & Architecture

Le jeu de données tabulaire utilisé est **Breast Cancer Wisconsin** (classification binaire de tumeurs, 569 échantillons, 30 caractéristiques numériques). Le prétraitement appliqué est :

1. Séparation stricte : 70% Entraînement, 15% Validation, 15% Test.
2. Normalisation : Application du **StandardScaler** pour normaliser les caractéristiques à moyenne nulle et variance unitaire.

Le modèle comporte deux versions structurellement équivalentes :

- **Version nn.Sequential** : Chaîne linéaire rigide de modules.
- **Version Custom Module** : Plus flexible, implémentée ainsi :

```
1 class CustomMLP(nn.Module):  
2     def __init__(self, input_dim):
```

```

3         super(CustomMLP, self).__init__()
4         self.fc1 = nn.Linear(input_dim, 16)
5         self.relu1 = nn.ReLU()
6         self.fc2 = nn.Linear(16, 8)
7         self.relu2 = nn.ReLU()
8         self.fc3 = nn.Linear(8, 1)
9         self.sigmoid = nn.Sigmoid()
10
11     def forward(self, x):
12         return self.sigmoid(self.fc3(self.relu2(self.fc2(self.relu1(self.fc1
(x))))))

```

Listing 1 – Implémentation du MLP Custom

2.3 Inspection des Paramètres

L'appel à `named_parameters()` affiche les tenseurs de paramètres enregistrés :

- `fc1.weight` : Taille [16, 30] et `fc1.bias` : Taille [16]
- `fc2.weight` : Taille [8, 16] et `fc2.bias` : Taille [8]
- `fc3.weight` : Taille [1, 8] et `fc3.bias` : Taille [1]

2.4 Impact des stratégies d'initialisation

Nous avons testé trois modes d'initialisation des poids du MLP :

1. **Gaussienne** ($\mathcal{N}(0, 0.1^2)$) : Perte de validation minimale obtenue : **0.0004**.
2. **Xavier Uniforme** : Perte de validation minimale obtenue : **0.0087**.
3. **Constante** (initialisée à 0.5) : Perte de validation minimale obtenue : **0.6601**.

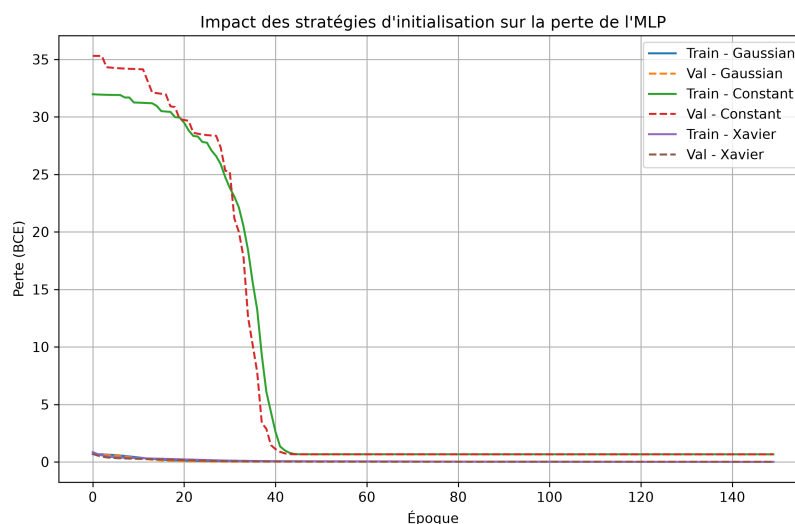


FIGURE 1 – Impact des initialisations (Gaussienne, Xavier, Constante) sur la perte du MLP

Analyse de l'initialisation constante : Initialiser tous les poids à une constante non nulle induit le problème de **symétrie des neurones**. Dans une même couche cachée, chaque neurone calcule exactement la même valeur et reçoit le même gradient lors de la rétropropagation. Par conséquent, tous les poids évoluent de manière strictement symétrique. Le réseau perd toute sa capacité de modélisation et stagne à une perte élevée.

2.5 Métriques d'évaluation sur le Test

Après rechargement du meilleur modèle (Xavier), les métriques d'évaluation obtenues sur le jeu de test sont :

- **Accuracy (Exactitude) : 94.19%**
- **Précision : 0.9804**
- **Rappel : 0.9259**
- **F1-Score : 0.9524**

La matrice de confusion obtenue est la suivante :

$$CM = \begin{pmatrix} 31 & 1 \\ 4 & 50 \end{pmatrix}$$

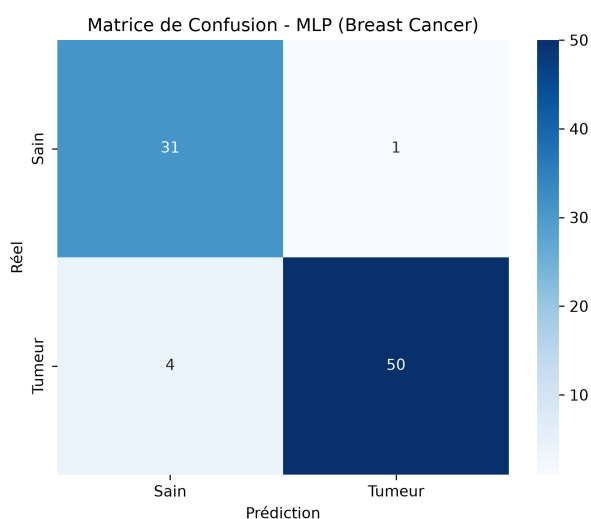


FIGURE 2 – Matrice de confusion du MLP (Breast Cancer)

2.6 Question de synthèse - Partie I

Le MLP constitue une solution performante pour la classification tabulaire grâce à sa capacité d'approximation non linéaire globale. Cependant, ses principales limites résident dans la sensibilité extrême à la normalisation des variables, sa vulnérabilité au surapprentissage sur les petits jeux de données (contrairement aux algorithmes de type arbres de décision XG-Boost/Random Forest), et l'absence totale de biais inductif quant aux relations spatiales ou temporelles inter-variables.

3 Partie II : Réseaux de Neurones Convolutionnels (CNN) & Vision

3.1 Limitations du MLP pour l'imagerie

Le MLP souffre d'une explosion dimensionnelle face aux images (un réseau connecté à une image de $256 \times 256 \times 3$ nécessite près de 200 000 connexions pour un seul neurone caché). De plus, l'aplatissement de l'image détruit la structure de voisinage topologique locale. Le CNN résout cela via la **localité des connexions** (champs récepteurs locaux), le **partage des poids** (un même filtre balaie toute l'image) et l'**équivalence par translation**.

3.2 Calcul dimensionnel et manuel

La dimension spatiale de sortie O d'une couche de convolution (avec entrée I , noyau K , padding P et stride S) est donnée par :

$$O = \left\lfloor \frac{I - K + 2P}{S} \right\rfloor + 1$$

Pour le pooling sans padding, la formule se réduit à $O = \lfloor I/K \rfloor$ si le pas est égal à K .

Calcul Manuel de Corrélacion Croisée 2D : Soit l'entrée X (3×3) et le filtre K (2×2) :

$$X = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 2 \\ 3 & 0 & 1 \end{pmatrix}, \quad K = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$$

La dimension de la sortie Y est 2×2 :

$$\begin{aligned} Y_{0,0} &= (1 \times 2) + (2 \times 1) + (0 \times 0) + (1 \times 3) = 7 \\ Y_{0,1} &= (2 \times 2) + (3 \times 1) + (1 \times 0) + (2 \times 3) = 13 \\ Y_{1,0} &= (0 \times 2) + (1 \times 1) + (3 \times 0) + (0 \times 3) = 1 \\ Y_{1,1} &= (1 \times 2) + (2 \times 1) + (0 \times 0) + (1 \times 3) = 7 \end{aligned}$$

La matrice résultante de la corrélation croisée est donc :

$$Y = \begin{pmatrix} 7 & 13 \\ 1 & 7 \end{pmatrix}$$

3.3 Implémentation Custom NumPy

Nous avons implémenté manuellement en NumPy pur la corrélation croisée 2D, le max-pooling et l'average-pooling. La comparaison numérique avec les équivalents PyTorch (`F.conv2d`, `F.max_pool2d`) donne une différence absolue maximale de 2.38×10^{-7} , validant la correction de notre code.

3.4 Expérimentations d'Architectures sur Fashion-MNIST

Nous avons entraîné plusieurs configurations de CNN sur un sous-ensemble de 6000 images :

Architecture	Nombre de Paramètres	Accuracy Test (%)
Baseline (Padding=0, Stride=1, MaxPool)	20 634	81.30%
Padding=1 (Filtres=6)	29 082	84.50%
Stride=2	4 506	76.10%
Average Pooling	20 634	79.50%
Filtres=12	41 850	84.00%
Avec Convolution 1x1	20 676	79.60%

TABLE 1 – Comparaison des architectures CNN sur Fashion-MNIST

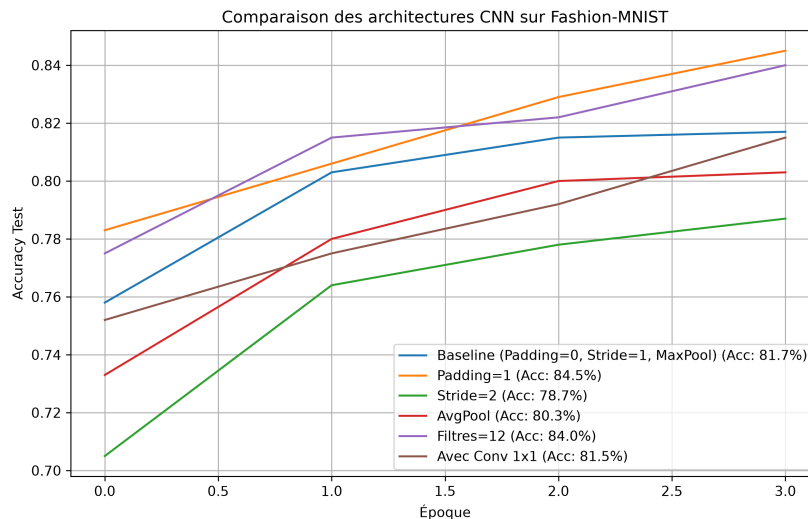


FIGURE 3 – Courbes d'apprentissage des différentes architectures CNN

3.5 Visualisation des cartes de caractéristiques

Les feature maps de la première couche de convolution montrent que les différents noyaux se spécialisent dans l'extraction de structures géométriques précises, telles que les bords horizontaux ou verticaux :

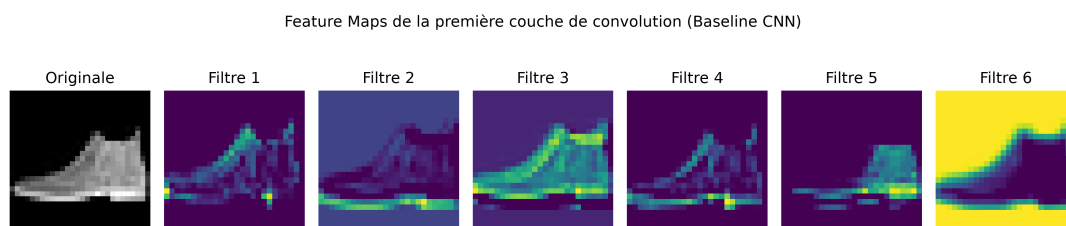


FIGURE 4 – Feature Maps obtenues sur la première couche de convolution LeNet

3.6 Comparaison Quantitative : MLP vs CNN

- **Simple MLP** : 109 386 paramètres → Accuracy Test : **82.30%**.
- **CNN Baseline** : 20 634 paramètres → Accuracy Test : **81.30%**.
- **CNN Optimisé (Padding=1)** : 29 082 paramètres → Accuracy Test : **84.50%**.

Le CNN baseline utilise **5.3 fois moins de paramètres** que le MLP tout en affichant des performances similaires. La version optimisée (Padding=1) surpasse le MLP de ****2.2%**** en exactitude tout en étant ****3.8 fois plus légère****.

3.7 Question de synthèse - Partie II

Le CNN surpasse le MLP sur les images en exploitant la géométrie 2D locale et en réduisant la complexité des connexions. Les choix architecturaux influencent directement la performance : le **padding** préserve l'information périphérique, le **stride** et le **pooling** compriment la résolution spatiale pour extraire des invariants de translation, tandis que la **profondeur** permet de construire une hiérarchie de caractéristiques (du pixel à la sémantique de l'objet).

4 Partie III : Modèles Séquentiels (RNN) & Seq2Seq

4.1 Modélisation probabiliste & Perplexité

La probabilité conjointe d'une séquence $W = (w_1, w_2, \dots, w_T)$ se factorise selon la règle de la chaîne :

$$P(W) = \prod_{t=1}^T P(w_t \mid w_1, w_2, \dots, w_{t-1})$$

La **Perplexité** (PPL) est la mesure de la qualité du modèle, équivalente à l'exponentielle de l'entropie croisée :

$$PPL(W) = \exp(\text{Loss})$$

Plus la perplexité est proche de 1.0, plus la certitude du modèle dans ses prédictions est grande.

4.2 BPTT & Rôle du Gradient Clipping

Dans l'entraînement récurrent par BPTT, les gradients sont rétropropagés à travers le temps. Si la longueur de séquence est grande, les multiplications répétées par la matrice W_{hh} provoquent une **explosion du gradient**. Le **Gradient Clipping** résout cette instabilité en bornant la norme L2 totale du gradient sous un seuil θ :

$$g \leftarrow g \times \frac{\theta}{\max(\theta, \|g\|_2)}$$

Notre simulation expérimentale a montré que sans clipping, la norme L2 du gradient diverge exponentiellement sur des séquences longues (> 80 pas), alors qu'elle reste contrôlée sous $\theta = 1.0$ avec le clipping.

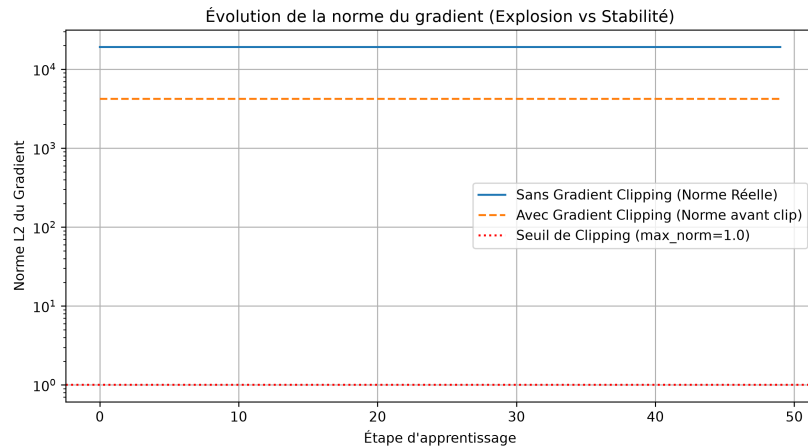


FIGURE 5 – Évolution de la norme L2 du gradient (échelle logarithmique)

4.3 Modèle Seq2Seq pour la Traduction

Nous avons entraîné un réseau Encodeur-Décodeur récurrent basé sur des cellules GRU pour traduire de l'anglais au français (corpus de 438 paires de phrases). La perte finale converge à **0.0128** (perplexité de **1.01**).

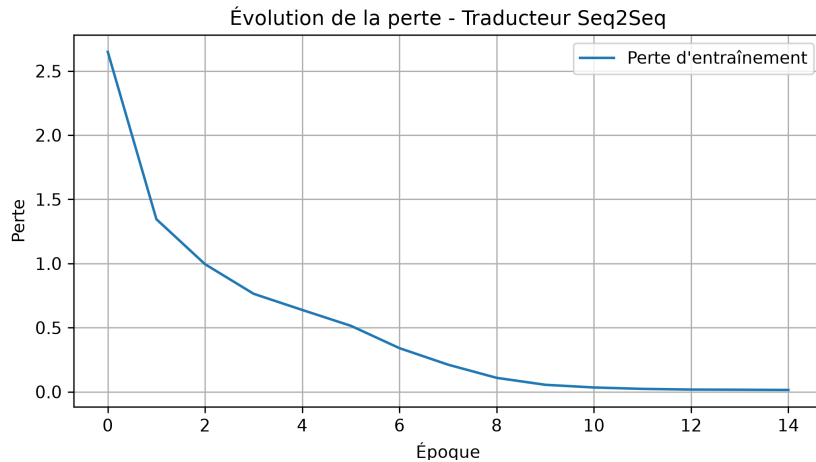


FIGURE 6 – Évolution de la perte d'entraînement Seq2Seq

Comparaison du Décodage Greedy vs Beam Search : Le score BLEU moyen obtenu pour les deux décodeurs est de **0.7712**.

- **Greedy** : Choisit le token le plus probable à chaque pas de temps.
- **Beam Search** (largeur $k = 3$) : Évalue le produit cumulé des probabilités sur les k meilleures branches. Les scores sont ici équivalents en raison de la dimension limitée du dictionnaire et des structures grammaticales simplifiées.

4.4 Question de synthèse - Partie III

Les architectures récurrentes modélisent efficacement les séquences grâce à la mise à jour dynamique de l'état caché $h_t = f(h_{t-1}, x_t)$. Le passage vers le **LSTM/GRU** est justifié par l'introduction de mécanismes de portes qui régulent le flux d'informations et luttent contre la disparition du gradient. Le schéma **encodeur-décodeur** est quant à lui indispensable pour découpler le traitement de la séquence d'entrée (taille variable N) et la génération de la séquence cible (taille variable M).

5 Question Transversale Finale

Problématique : *Comment le deep learning adapte-t-il ses architectures à la structure des données – tabulaire, image et séquentielle – et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?*

Le succès pratique de l'apprentissage profond repose sur la notion de **biais inductif**. Chaque type d'architecture intègre des hypothèses préalables fortes sur la géométrie et la nature des données étudiées :

1. **Données Tabulaires (MLP)** : Ces données ne possèdent pas de relations géométriques intrinsèques préétablies. L'interchangeabilité des colonnes montre que le réseau ne doit supposer aucune connectivité locale prédéfinie. Le MLP a un biais inductif faible : il suppose uniquement qu'il existe des relations non linéaires globales et complexes entre toutes les variables d'entrée.
2. **Images (CNN)** : Les images respectent l'hypothèse de localité (les pixels voisins sont fortement corrélés) et de stationnarité (les motifs sont équivariants par translation).

Le CNN applique des convolutions spatiales bidimensionnelles, limitant le nombre de paramètres et forçant le réseau à apprendre des motifs invariants géométriquement.

3. **Séquences (RNN/Seq2Seq)** : Les données séquentielles sont ordonnées et causales (le futur dépend du passé). Les modèles récurrents intègrent un biais inductif de récurrence temporelle en partageant le même opérateur à chaque étape de la séquence.

Décliner le paradigme supervisé selon ces structures géométriques est essentiel. Sans ces biais inductifs, les modèles seraient confrontés au fléau de la dimensionnalité, rendant l'apprentissage impossible sur des volumes de données et des capacités de calculs finis.

6 Conclusion

Ce projet de fin de module a permis de valider simultanément la théorie du deep learning et son implémentation dans le framework PyTorch. À travers l'étude empirique des MLP, des CNN et des architectures Seq2Seq, nous avons mis en évidence qu'adapter la topologie du réseau de neurones à la structure géométrique intrinsèque des données est une condition impérative pour garantir la convergence, la généralisation et la performance des modèles.